

Shortest Path with Negative Weights: Bellman
Ford's Algorithm
(ADA 2023, CSE 222)

Diptapriyo Majumdar

March 27, 2023

Dijkstra's Algorithm

Initialize $d[s] \leftarrow 0$;

For all $v \in V(G) \setminus \{s\}$, $d[v] \leftarrow \infty$;

$S \leftarrow \emptyset$ and $Q \leftarrow V(G)$;

while $Q \neq \emptyset$ **do**

$u \leftarrow \text{ExtractMin}(Q)$;

$S \leftarrow S \cup \{u\}$;

for each $v \in N_G(u)$ **do**

if $d[v] > d[u] + \text{weight}(u, v)$ **then**

$d[v] = d[u] + \text{weight}(u, v)$;

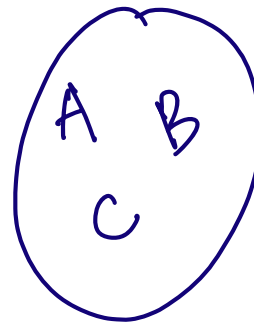
$\text{prev}(v) \leftarrow u$;

end

end

end

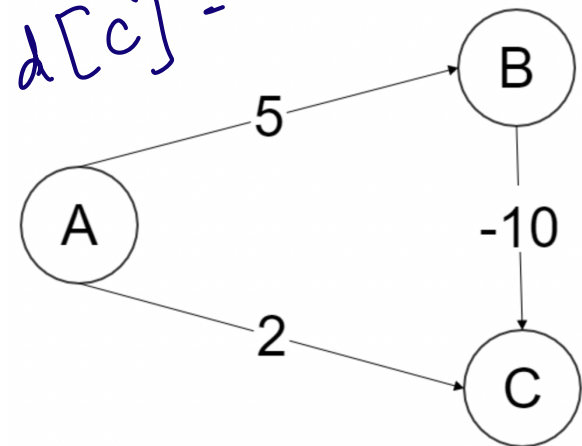
$Q = \{A, B, C\}$
 $\downarrow$
 $Q = \{B, C\}$
 $\downarrow$
 $Q = \{B\}$
 S



$d[A] = 0$

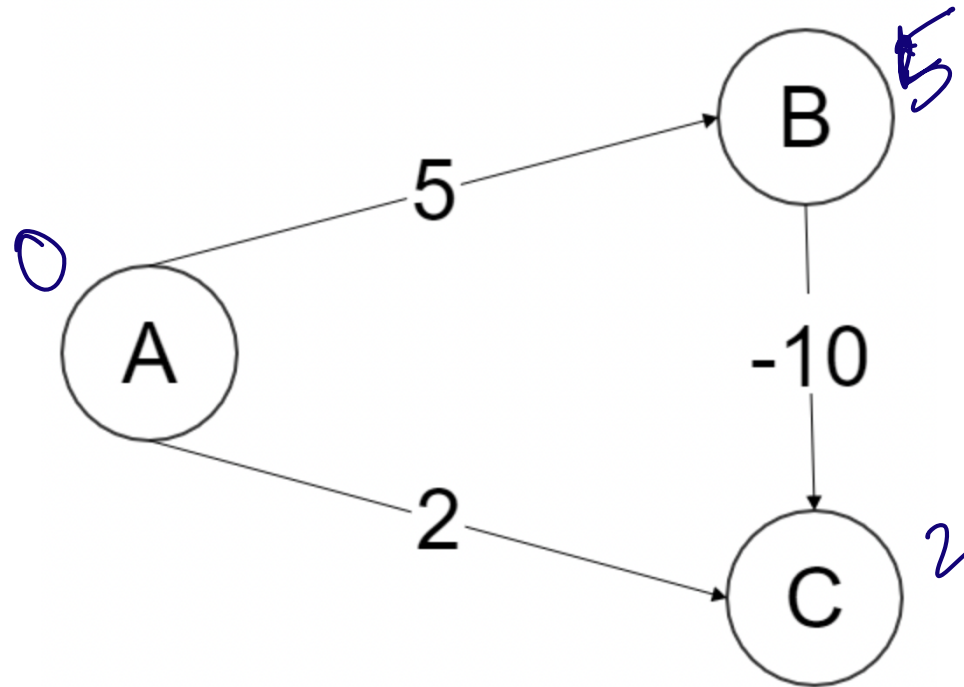
$A = s$
 $d[B] = 5$

$d[C] = 2$



Presence of negative weighted edge

- **Input:** A directed graph $G = (V, E)$, every edge (u, v) has a $\text{weight}(u, v) \in \mathbb{R}$ and $s \in V(G)$.
- **Goal:** Paths from s to all the vertices of G with smallest total weight.



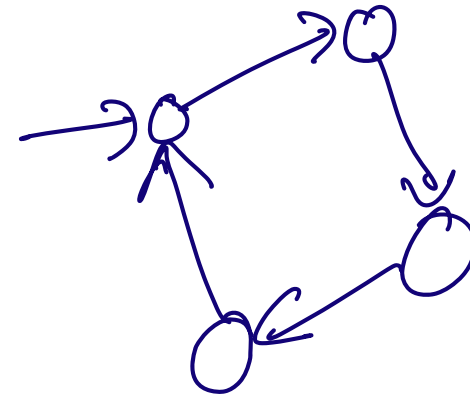
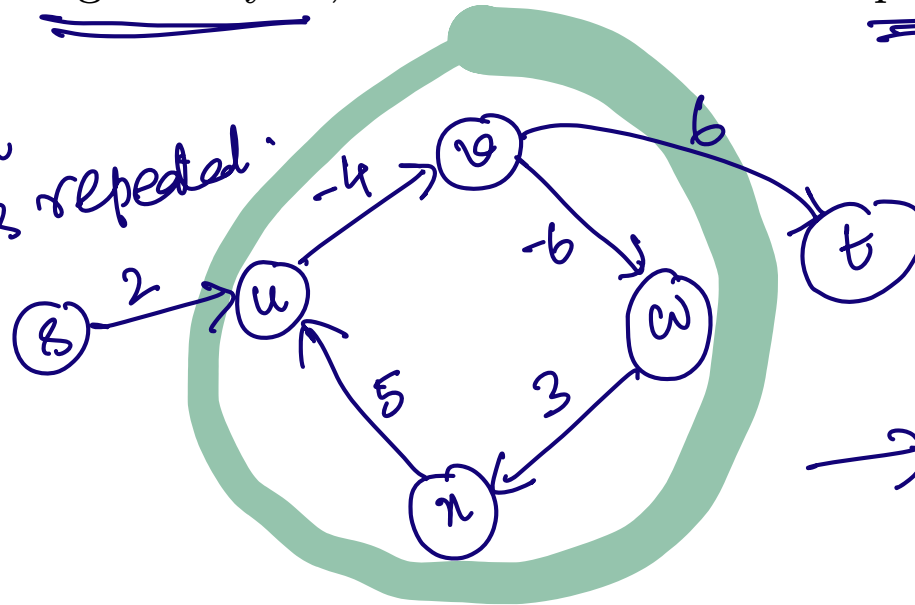
Outline

- ① Bellman Ford's Algorithm
- ② Application of Bellman Ford

Algorithm

- **Assumption:** G has no negative cycle.
- If G has no negative cycle, then there is a shortest path from s to t that is simple.

simple path
 $\equiv$ no vertex is repeated.

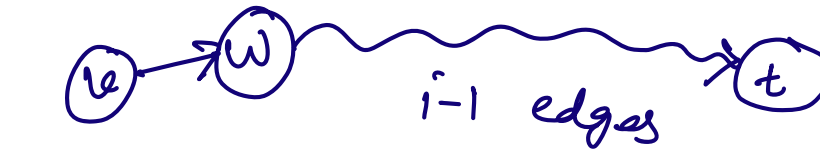


Consider any directed cycle C .

$\sum_{(u,v) \in C} \text{weight}(u,v) > 0$. Hence, using all the edges is of no use.

Dynamic Programming

- If G has no negative cycle, then there is a shortest path from s to t that is simple.
- Let $OPT(i, v)$ is the minimum cost of a v - t path (a path from v to t) using at most i edges.
- Our original problem is to compute $OPT(n - 1, s)$.
- **Lemma:** Let P be an optimal path from v to t . Then the following statements hold true. *shortest path*
 - If the path uses at most $i - 1$ edges, then $OPT(i - 1, v) = OPT(i, v)$.
 - If the path uses i edges and the first edge is (v, w) , then $OPT(i, v) = OPT(i - 1, w) + \text{weight}(v, w)$.



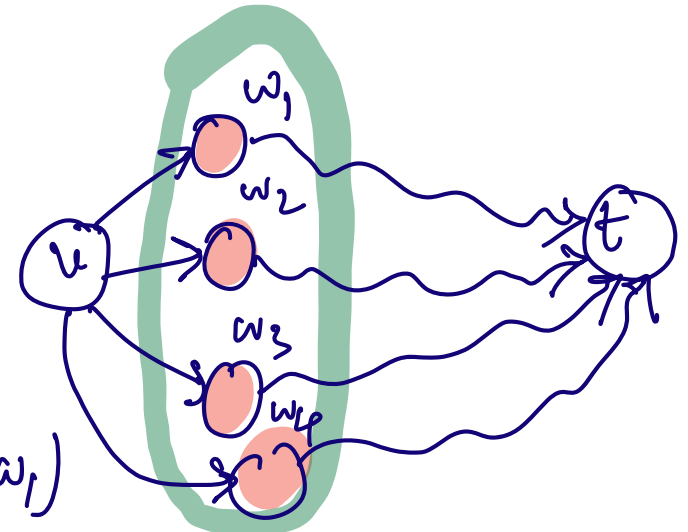
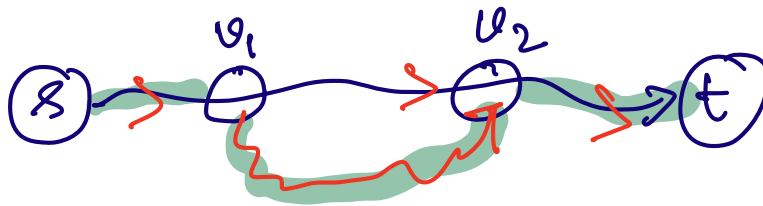
- **Lemma:** Let P be an optimal path from v to t . Then the following statements hold true.

- If the path uses at most $i - 1$ edges, then $OPT(i - 1, v) = OPT(i, v)$.
- If the path uses i edges and the first edge is (v, w) , then $OPT(i, v) = OPT(i - 1, w) + \text{weight}(v, w)$.

- If $i > 0$, then

$$OPT(i, v) = \min \left\{ OPT(i - 1, v), \min_{w: v \in N(w)} \left(OPT(i - 1, w) + \text{weight}(v, w) \right) \right\}$$

Shortest Path satisfies optimal substructure property.



$$\min \left\{ \begin{array}{l} OPT(i - 1, v) \\ OPT(i - 1, w_1) + \text{weight}(v, w_1) \\ OPT(i - 1, w_2) + \text{weight}(v, w_2) \\ OPT(i - 1, w_3) + \text{weight}(v, w_3) \\ OPT(i - 1, w_4) + \text{weight}(v, w_4) \end{array} \right\}$$

For all $v \in V(G) \setminus \{t\}$, $M[0, v] \leftarrow \infty$;

$M[0, t] \leftarrow 0$;

for $i = 1, \dots, n-1$ do

for each $v \in V(G)$ in any order do

Compute $M[i, v]$ using the recurrence;

end

end

$M[n-1, s]$;

Shortest path weight
from s to t using at most

For $i > 0$, $M[i, v] = \min \left\{ M[i-1, v], \min_{\substack{w: v \in N(w) \\ w \in N(v)}} (M[i-1, w] + \text{weight}(v, w)) \right\}$

$n-1$ edges.

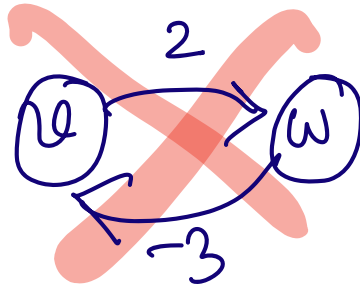
$M[1, v]$

then

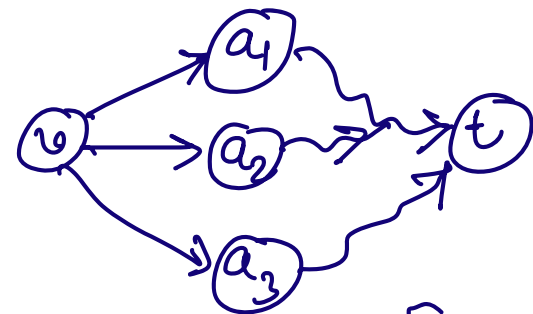
$M[2, v]$

, ...

$M[n-1, v]$



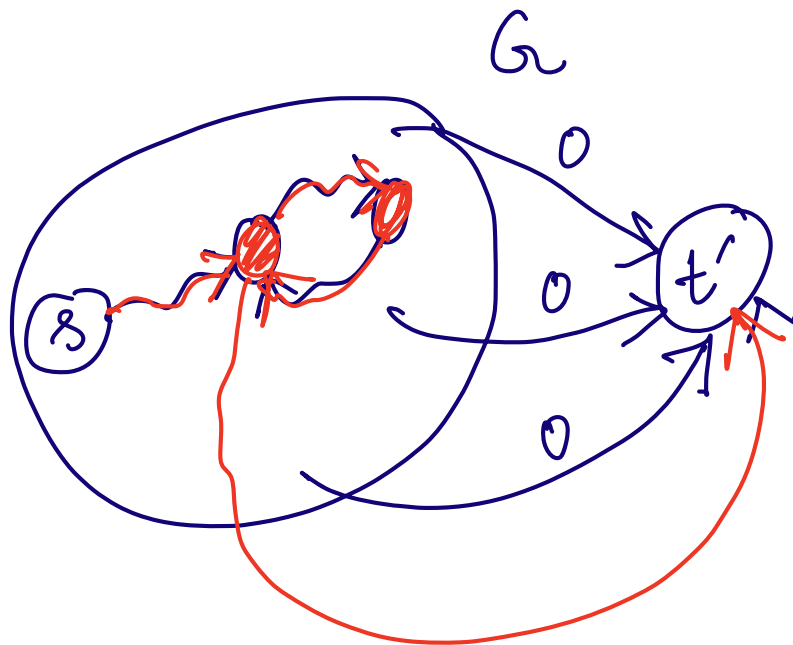
$M[n, v]$



$$M[i, v] = \min \begin{cases} M[i-1, v] \\ M[i-1, a_1] + w(v, a_1) \\ M[i-1, a_2] + w(v, a_2) \\ M[i-1, a_3] + w(v, a_3) \end{cases}$$

Detecting Negative Cycles

- **Augment the graph:** Add a new node t' , and for all $v \in V(G)$, make (v, t') an edge. This graph is G'
- G' has a negative cycle C such that there is a path from C to t' if and only if G has a negative cycle.



- What will happen to $OPT(i, v)$ for all $i \geq n$ when there is no negative cycle in G ?
- If there is a negative cycle in G , then how about what happens to the following quantity?

$$\lim_{i \rightarrow \infty} OPT(i, v)$$

- What will happen to $OPT(i, v)$ for all $i \geq n$ when there is no negative cycle in G ?

$$OPT(n, v) = OPT(n-1, v) = OPT(n+2, v)$$

- If there is a negative cycle in G , then how about what happens to the following quantity?

$$\lim_{i \rightarrow \infty} OPT(i, v)$$

$$n = |V|.$$

- If there is no negative cycle in G , then $OPT(i, v) = OPT(n-1, v)$ for all $i \geq n$.
- There is no negative cycle in G with a path to t if and only if for all vertices $v \in V(G)$, we have $OPT(n, v) = OPT(n-1, v)$.

- There is no negative cycle in G with a path to t if and only if for all vertices $v \in V(G)$, we have $OPT(n, v) = OPT(n - 1, v)$.

- If G has n nodes and $OPT(n, v) \neq OPT(n - 1, v)$, then a path from v to t of total weight $OPT(n, v)$ contains a negative cycle.

Outline

- ① Bellman Ford's Algorithm
- ② Application of Bellman Ford

Arbitrage

Suppose 1 USD has bought 0.82 EUR, 1 EUR bought 129.7 Japanese Yen, 1 Japanese Yen bought 12 Turkish Lira, and 1 Turkish Lira bought 0.0008 USD.

- **Input:** A collection of n currencies and their mutual exchange rates, e.g. $R[c_i, c_j]$ is the exchange rate from currency c_i to currency c_j .
- **Question:** Is there a sequence of currencies $c_i^1, \dots, c_i^j$ such that the following holds true?

$$R[c_{i,1}, c_{i,2}] \cdot R[c_{i,2}, c_{i,3}] \cdots R[c_{i,j-1}, c_{i,j}] \cdot R[c_{i,j}, c_{i,1}] > 1$$

$$1 \text{ USD} \rightarrow 0.82 \text{ EUR} \rightarrow \begin{matrix} 0.82 \times 129.7 \\ \text{Yen} \end{matrix} \rightarrow \begin{matrix} 0.82 \times 129.7 \\ \times 12 \end{matrix}$$

$$1.03 \text{ USD} \leftarrow 0.82 \times 129.7 \times 12 \times 0.0008 \leftarrow$$

$$\begin{aligned} & \log(R[c_{i,1}, c_{i,2}]) + \log(R[c_{i,2}, c_{i,3}]) + \dots + \log(R[c_{i,j}, c_{i,1}]) \\ &= \log(R[c_{i,1}, c_{i,2}] R[c_{i,2}, c_{i,3}] \dots R[c_{i,j}, c_{i,1}]) \\ & > 0 \end{aligned}$$

$$\log \frac{1}{R} = -\log R$$

Take $\log \left(\frac{1}{R[c_i, c_j]} \right)$.

